

# Documento de Apresentação: Sistema Simplificado de Ponto v1.0

## 1. Introdução

**Objetivo do Projeto:** Desenvolver um sistema digital, moderno e funcional para o registro de ponto de funcionários, substituindo processos manuais e centralizando os dados de forma segura.

**Versão Atual:** v1.0 - Esta versão representa o Produto Mínimo Viável (MVP), entregando as funcionalidades essenciais de cadastro, autenticação, registro de ponto e gestão básica de usuários.

**Objetivo do Documento:** Apresentar as funcionalidades implementadas na v1.0, detalhar a arquitetura de software adotada e explicar as tecnologias utilizadas, justificando as escolhas e descrevendo como cada uma foi aplicada no projeto.

## 2. Visão Geral do Projeto (Funcionalidades v1.0)

O sistema opera com dois perfis de usuário principais: **Funcionário** e **Administrador**.

### Funcionalidades do Funcionário:

- **Login:** Autenticação segura utilizando Matrícula e Senha.
- **Registro de Ponto:** Capacidade de registrar horários de Entrada e Saída. O sistema aplica regras de negócio para garantir a sequência correta (uma Entrada deve preceder uma Saída, não permite duas Entradas abertas).
- **Logout:** Encerramento seguro da sessão.

### Funcionalidades do Administrador:

- **Login:** Autenticação segura utilizando Matrícula e Senha (distinta do login de funcionário).
- **Visualizar Funcionários:** Acesso a uma lista de todos os usuários com perfil de Funcionário, exibindo Nome, Matrícula e Status (Ativo/Inativo).
- **Desativar Funcionário:** Capacidade de alterar o status de um funcionário para "Inativo", impedindo futuros logins deste usuário, mas preservando seu histórico.
- **Logout:** Encerramento seguro da sessão.

### 3. Arquitetura de Software

O sistema foi desenvolvido seguindo uma arquitetura moderna e desacoplada:

- **API Backend + Frontend Separado:** Optou-se por uma API RESTful no backend e uma interface de usuário (frontend) separada. Isso permite maior flexibilidade, escalabilidade e a possibilidade de criar diferentes interfaces (web, mobile) consumindo a mesma lógica de negócio.
- **Clean Architecture (Backend):** A API foi estruturada em camadas com responsabilidades bem definidas:
  - **Domain:** Contém as entidades de negócio (Usuario, RegistroPonto) e as regras mais essenciais. É o núcleo independente da aplicação.
  - **Application:** Orquestra os casos de uso (Serviços) e define as interfaces (contratos) para as outras camadas. Contém a lógica de aplicação.
  - **Infrastructure:** Implementa os detalhes técnicos, como acesso ao banco de dados (Repositórios), serviços de autenticação, hashing, etc. Depende da camada de Application (para implementar as interfaces).
  - **Api (Presentation):** Expõe as funcionalidades da camada de Application através de endpoints HTTP (Controllers). É a porta de entrada da aplicação.

### 4. Tecnologias Utilizadas

A escolha das tecnologias visou atender aos requisitos do projeto, priorizando robustez, segurança e boas práticas de desenvolvimento.

- **C# e .NET 8:**
  - **O que é:** Linguagem de programação moderna e framework de desenvolvimento da Microsoft.
  - **Por que foi escolhida?** Atendeu à preferência inicial; plataforma robusta, de alta performance, com forte tipagem; excelente ecossistema e ferramentas; suporte nativo a APIs, Swagger e testes.
  - **Como foi usada no projeto?** Usado para construir toda a lógica da API backend, incluindo Controllers, Services, Repositórios e Entidades, seguindo a Clean Architecture.
- **PostgreSQL:**
  - **O que é:** Sistema de gerenciamento de banco de dados relacional open-source.
  - **Por que foi escolhida?** Banco de dados robusto, confiável, escalável e amplamente utilizado no mercado. Adequado para armazenar dados de usuários e registros de forma segura.

- **Como foi usada no projeto?** Usado para persistir os dados das tabelas Usuarios e RegistrosPonto. O banco roda em um container Docker gerenciado pelo Docker Compose.
- **Entity Framework Core 8:**
  - **O que é:** ORM (Object-Relational Mapper) padrão para .NET.
  - **Por que foi escolhida?** Simplifica o acesso e manipulação de dados no banco, abstraindo a escrita de SQL; integra-se perfeitamente com C#/.NET; gerencia o schema do banco via Migrations.
  - **Como foi usada no projeto?** Definiu o ApplicationDbContext para mapear as entidades Usuario e RegistroPonto para as tabelas; usado nos Repositórios para realizar operações CRUD (Adicionar, Obter, Atualizar) usando LINQ; gerenciou a criação inicial das tabelas.
- **API RESTful:**
  - **O que é:** Estilo arquitetural para construção de web services usando HTTP.
  - **Por que foi escolhida?** Requisito do projeto para desacoplar backend e frontend; padrão de mercado para comunicação entre sistemas web.
  - **Como foi usada no projeto?** Implementada usando ASP.NET Core Controllers ([ApiController], [Route], [HttpGet], [HttpPost], [HttpPatch]). Os dados são trocados no formato JSON.
- **JWT (JSON Web Tokens):**
  - **O que é:** Padrão aberto (RFC 7519) para criar tokens de acesso seguros.
  - **Por que foi escolhida?** Solução padrão e stateless para autenticação em APIs RESTful, permitindo que o frontend envie credenciais apenas uma vez (no login).
  - **Como foi usada no projeto?** Usado o pacote Microsoft.AspNetCore.Authentication.JwtBearer. Configurado no Program.cs para validar tokens. Um TokenService foi criado para gerar tokens JWT assinados com uma chave secreta após o login bem-sucedido. Os endpoints protegidos usam [Authorize].
- **BCrypt.Net-Next:**
  - **O que é:** Biblioteca .NET para hashing de senhas usando o algoritmo BCrypt.
  - **Por que foi escolhida?** Requisito de segurança (RNF03) para armazenar senhas de forma segura. BCrypt é o padrão da indústria, pois inclui "salting" automaticamente.
  - **Como foi usada no projeto?** Um PasswordHasherService foi criado. HashPassword é usado ao criar um usuário. VerifyPassword é usado durante o login para comparar a senha fornecida com o hash armazenado.

- **Swagger (Swashbuckle):**
  - **O que é:** Ferramenta para documentar e testar APIs RESTful.
  - **Por que foi escolhida?** Requisito do projeto; facilita o entendimento e teste da API; integração nativa com templates .NET (via Swashbuckle).
  - **Como foi usada no projeto?** Configurado no Program.cs (AddSwaggerGen, UseSwagger, UseSwaggerUI). A interface do Swagger, acessível em /swagger, permite visualizar e executar todos os endpoints, incluindo a autenticação JWT.
- **HTML / CSS / JavaScript:**
  - **O que é:** Tecnologias padrão para construção de interfaces web.
  - **Por que foi escolhida?** Escolhidas na forma "vanilla" (sem frameworks) para manter a simplicidade do frontend no escopo do MVP e atender aos requisitos de design minimalista.
  - **Como foi usada no projeto?** index.html define a estrutura base. style.css implementa o design (tema escuro, layout). app.js gerencia a renderização dinâmica das "telas" (Seleção, Login, Dashboards) e a comunicação com a API usando fetch.
- **Docker / Docker Compose:**
  - **O que é:** Plataforma de containerização e ferramenta de orquestração.
  - **Por que foi escolhida?** Requisito do projeto; garante consistência entre ambientes de desenvolvimento e produção; simplifica o setup e a execução do projeto (API + Banco).
  - **Como foi usada no projeto?** Um Dockerfile define como construir a imagem da API .NET. O docker-compose.yml define os serviços api e db (PostgreSQL), gerenciando a rede interna, portas e persistência de dados do banco (volumes).
- **Git / GitHub:**
  - **O que é:** Sistema de controle de versão distribuído e plataforma de hospedagem.
  - **Por que foi escolhida?** Prática padrão de desenvolvimento para versionamento, backup e colaboração. Requisito do usuário.
  - **Como foi usada no projeto?** O projeto foi inicializado com Git, um .gitignore foi configurado, o código foi commitado e enviado para um repositório no GitHub. A versão v1.0 foi marcada com uma tag Git.
- **xUnit / Moq:**
  - **O que é:** Framework de testes de unidade e biblioteca de mocking para .NET.
  - **Por que foi escolhida?** Requisito do projeto (mínimo 5 testes); garante a qualidade e a corretude da lógica de negócio; facilita refatorações futuras.

- **Como foi usada no projeto?** Um projeto `SistemaPonto.Tests` foi criado. Foram implementados 5 testes de unidade (3 para `RegistroPontoService`, 2 para `UsuarioService`) usando `xUnit` para a estrutura e `Moq` para isolar dependências (mocking de repositórios). Os testes são executados com `dotnet test`.

## 5. Conclusão e Próximos Passos

A versão v1.0 do Sistema Simplificado de Ponto entrega um núcleo funcional e seguro, construído sobre uma arquitetura moderna e tecnologias robustas. O projeto atende a todos os requisitos essenciais definidos para o MVP.

Os próximos passos incluem:

- Finalizar a interface do Administrador (conectar o botão "Desativar").
- Revisar e finalizar a documentação (`Readme.md`).
- Planejar e executar a implantação (deployment) em um ambiente de produção.
- Implementar funcionalidades futuras (cálculo de horas, reativação de usuários, etc.) em versões subsequentes (v1.1, v2.0).